Building a Perceptron Classifier with NumPy

An introductory guide to implementing a **Single-Layer Perceptron (SLP)** from scratch using Python and NumPy. This walkthrough covers fundamental artificial neural network principles by training a single linear neuron to solve a boolean logic problem (**AND gate**).

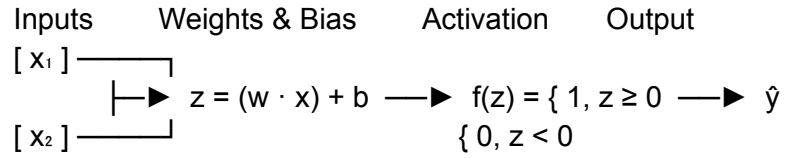
Technical Summary

The Perceptron, proposed by Frank Rosenblatt, serves as the foundation for modern deep learning models. It computes a linear combination of input features and applies a threshold function to make binary classification decisions.

Key Objectives

- Construct a vector-based Perceptron class from scratch using **NumPy**.
- Implement a step threshold function for binary decision-making.
- Apply the standard Perceptron Delta Rule for optimization.
- Train and evaluate the model on linearly separable binary data (**AND logic operation**).

Mathematical Foundations



1. **Weighted Sum (Pre-activation):**
   $$z = \sum_{i=1}^{n} w_i x_i + b = \mathbf{w}^T \mathbf{x} + b$$
2. **Heaviside Step Function (Activation):**
   $$\hat{y} = \begin{cases} 1 & \text{if } z \ge 0 \\ 0 & \text{if } z < 0 \end{cases}$$
3. **Weight & Bias Adjustment Rules:**
   $$\Delta \mathbf{w} = \alpha \cdot (y - \hat{y}) \cdot \mathbf{x}$$
   $$\Delta b = \alpha \cdot (y - \hat{y})$$
   $$\mathbf{w}^{(t+1)} = \mathbf{w}^{(t)} + \Delta \mathbf{w}$$
   $$b^{(t+1)} = b^{(t)} + \Delta b$$
   Where α represents the learning rate, y is the actual label, and $\hat{y}$ is the predicted output.

Dataset: Boolean AND Gate

The truth table serves as our feature matrix X and target vector y:

Feature 1 (x1)	Feature 2 (x2)	Target (y)

0	0	**0**
0	1	**0**
1	0	**0**
1	1	**1**

Python & NumPy Implementation

```
import numpy as np
```

```
def _step_function(x):
    """
    Step activation function.
    Returns 1 if x >= 0, otherwise returns 0.
    """
    return 1 if x >= 0 else 0


class Perceptron:
    """
    A simple Perceptron model for binary classification.
    """

    def __init__(self, input_size, learning_rate=0.1):
        """
        Initializes the Perceptron with random weights and bias.

        Args:
            input_size (int): The number of features in the input data.
            learning_rate (float): The learning rate for weight
updates.
        """
```

```
        # Initialize weights randomly to break symmetry, typically
small values
        # For simplicity, starting with zeros as in the original code,
but random is often better.
        self.weights = np.zeros(input_size)
        # Initialize bias to 0
        self.bias = 0
        # Set the learning rate
        self.lr = learning_rate
        print(f"Perceptron initialized with input_size={input_size},
learning_rate={self.lr}")

    def predict(self, x):
        """
        Makes a prediction for a single input vector x.

        Args:
            x (np.array): A single input feature vector.

        Returns:
            int: The predicted class (0 or 1).
        """
        # Calculate the weighted sum of inputs and add bias
        # z = w1*x1 + w2*x2 + ... + wn*xn + bias
        z = np.dot(x, self.weights) + self.bias
        # Apply the step activation function
        return _step_function(z)

    def train(self, X, y, epochs=10):
        """
        Trains the Perceptron model using the given training data.

        Args:
            X (np.array): Training input features (N_samples,
N_features).
            y (np.array): Target labels (N_samples,).
            epochs (int): The number of training iterations over the
entire dataset.
        """
        print(f"\nStarting training for {epochs} epochs...")
        for epoch in range(epochs):
            total_error = 0
            for i, (xi, target) in enumerate(zip(X, y)):
```

```
                # Make a prediction with current weights
                prediction = self.predict(xi)
                # Calculate the error
                error = target - prediction
                total_error += abs(error)

                # Update weights and bias based on the error
                # If prediction is wrong, adjust weights and bias
                if error != 0:
                    # Update weights: weights = weights + learning_rate
* error * input
                    self.weights += self.lr * error * xi
                    # Update bias: bias = bias + learning_rate * error
                    self.bias += self.lr * error
            print(f"Epoch {epoch + 1}/{epochs}, Total Error:
{total_error}, Weights: {self.weights}, Bias: {self.bias:.2f}")
            # If total_error is 0, it means all samples are correctly
classified
            if total_error == 0:
                print("Converged! No errors in this epoch.")
                break
        print("Training complete.")


if __name__ == "__main__":
    # --- 1. Define Training Data (AND gate example) ---
    # Input features (X) for the AND gate
    X_train = np.array([
        [0, 0],
        [0, 1],
        [1, 0],
        [1, 1]
    ])
    # Target labels (y) for the AND gate
    y_train = np.array([0, 0, 0, 1])

    print("--- Perceptron for AND Gate ---")
    print("Training Data (X):\n", X_train)
    print("Target Labels (y):", y_train)

    # --- 2. Initialize and Train the Perceptron ---
    # Create a Perceptron instance with 2 input features
    perceptron = Perceptron(input_size=2, learning_rate=0.1)
```

```
    # Train the perceptron for a specified number of epochs
    perceptron.train(X_train, y_train, epochs=10)

    print("\n--- Final Model Parameters ---")
    print(f"Final Weights: {perceptron.weights}")
    print(f"Final Bias: {perceptron.bias:.2f}")

    # --- 3. Test Predictions ---
    print("\n--- Predictions on Training Data ---")
    correct_predictions = 0
    for xi, target in zip(X_train, y_train):
        prediction = perceptron.predict(xi)
        print(f"Input: {xi}, Target: {target}, Predicted:
{prediction}")
        if prediction == target:
            correct_predictions += 1

    # --- 4. Calculate and Display Accuracy ---
    accuracy = (correct_predictions / len(y_train)) * 100
    print(f"\nAccuracy on training data: {accuracy:.2f}%")

    # You can also test with new data if needed
    # print("\n--- Testing with a new input (e.g., [0,1]) ---")
    # new_input = np.array([0, 1])
    # print(f"Input: {new_input}, Predicted:
{perceptron.predict(new_input)}")
```

output

```
--- Perceptron for AND Gate ---
Training Data (X):
 [[0 0]
 [0 1]
 [1 0]
 [1 1]]
Target Labels (y): [0 0 0 1]
Perceptron initialized with input_size=2, learning_rate=0.1

Starting training for 10 epochs...
Epoch 1/10, Total Error: 2, Weights: [0.1 0.1], Bias: 0.00
Epoch 2/10, Total Error: 3, Weights: [0.2 0.1], Bias: -0.10
Epoch 3/10, Total Error: 3, Weights: [0.2 0.1], Bias: -0.20
Epoch 4/10, Total Error: 0, Weights: [0.2 0.1], Bias: -0.20
Converged! No errors in this epoch.
Training complete.

--- Final Model Parameters ---
Final Weights: [0.2 0.1]
Final Bias: -0.20

--- Predictions on Training Data ---
Input: [0 0], Target: 0, Predicted: 0
Input: [0 1], Target: 0, Predicted: 0
Input: [1 0], Target: 0, Predicted: 0
Input: [1 1], Target: 1, Predicted: 1

Accuracy on training data: 100.00%
```